

Edit prompt

Hero

use this for hero section You are given a task to integrate an existing React component in the codebase

The codebase should support:

- shadcn project structure
- Tailwind CSS
- Typescript

If it doesn't, provide instructions on how to setup project via shadcn CLI, install Tailwind or Typescript.

Determine the default path for components and styles.

If default path for components is not /components/ui, provide instructions on why it's important to create this folder

Copy-paste this component to /components/ui folder:

```
``tsx
```

```
aether-flow-hero.tsx
```

```
"use client";
```

```
import React from 'react';
```

```
import { motion } from 'framer-motion';
```

```
import { ArrowRight, Zap } from 'lucide-react';
```

```
// A utility function for class names
```

```
const cn = (...classes) => classes.filter(Boolean).join(' ');
```

```
// The main hero component
```

```
const AetherFlowHero = () => {
```

```
  const canvasRef = React.useRef(null);
```

```
  React.useEffect(() => {
```

```
    const canvas = canvasRef.current;
```

```
    if (!canvas) return;
```

```
    const ctx = canvas.getContext('2d');
```

```
    let animationFrameId;
```

```
    let particles = [];
```

```
    const mouse = { x: null, y: null, radius: 200 };
```

// Moved Particle class definition here to avoid initialization errors

```
class Particle {
  constructor(x, y, directionX, directionY, size, color) {
    this.x = x;
    this.y = y;
    this.directionX = directionX;
    this.directionY = directionY;
    this.size = size;
    this.color = color;
  }

  draw() {
    ctx.beginPath();
    ctx.arc(this.x, this.y, this.size, 0, Math.PI * 2, false);
    ctx.fillStyle = this.color;
    ctx.fill();
  }

  update() {
    if (this.x > canvas.width || this.x < 0) {
      this.directionX = -this.directionX;
    }
    if (this.y > canvas.height || this.y < 0) {
      this.directionY = -this.directionY;
    }

    // Mouse collision detection
    if (mouse.x !== null && mouse.y !== null) {
      let dx = mouse.x - this.x;
      let dy = mouse.y - this.y;
      let distance = Math.sqrt(dx * dx + dy * dy);
      if (distance < mouse.radius + this.size) {
        const forceDirectionX = dx / distance;
        const forceDirectionY = dy / distance;
        const force = (mouse.radius - distance) / mouse.radius;
        this.x -= forceDirectionX * force * 5;
        this.y -= forceDirectionY * force * 5;
      }
    }

    this.x += this.directionX;
    this.y += this.directionY;
    this.draw();
  }
}
```

```

    }
  }

function init() {
  particles = [];
  let numberOfParticles = (canvas.height * canvas.width) / 9000;
  for (let i = 0; i < numberOfParticles; i++) {
    let size = (Math.random() * 2) + 1;
    let x = (Math.random() * ((innerWidth - size * 2) - (size * 2)) + size * 2);
    let y = (Math.random() * ((innerHeight - size * 2) - (size * 2)) + size * 2);
    let directionX = (Math.random() * 0.4) - 0.2;
    let directionY = (Math.random() * 0.4) - 0.2;
    let color = 'rgba(191, 128, 255, 0.8)'; // Brighter purple
    particles.push(new Particle(x, y, directionX, directionY, size, color));
  }
};

const resizeCanvas = () => {
  canvas.width = window.innerWidth;
  canvas.height = window.innerHeight;
  init();
};

window.addEventListener('resize', resizeCanvas);
resizeCanvas();

const connect = () => {
  let opacityValue = 1;
  for (let a = 0; a < particles.length; a++) {
    for (let b = a; b < particles.length; b++) {
      let distance = ((particles[a].x - particles[b].x) * (particles[a].x - particles[b].x))
        + ((particles[a].y - particles[b].y) * (particles[a].y - particles[b].y));

      if (distance < (canvas.width / 7) * (canvas.height / 7)) {
        opacityValue = 1 - (distance / 20000);

        let dx_mouse_a = particles[a].x - mouse.x;
        let dy_mouse_a = particles[a].y - mouse.y;
        let distance_mouse_a = Math.sqrt(dx_mouse_a*dx_mouse_a +
dy_mouse_a*dy_mouse_a);

        if (mouse.x && distance_mouse_a < mouse.radius) {
          ctx.strokeStyle = `rgba(255, 255, 255, ${opacityValue})`;
        } else {
          ctx.strokeStyle = `rgba(200, 150, 255, ${opacityValue})`;
        }
      }
    }
  }
};

```

```

    }

    ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.moveTo(particles[a].x, particles[a].y);
    ctx.lineTo(particles[b].x, particles[b].y);
    ctx.stroke();
  }
}
}
};

const animate = () => {
  animationFrameId = requestAnimationFrame(animate);
  // Set the background color inside the canvas draw loop
  ctx.fillStyle = 'black';
  ctx.fillRect(0, 0, innerWidth, innerHeight);

  for (let i = 0; i < particles.length; i++) {
    particles[i].update();
  }
  connect();
};

const handleMouseMove = (event) => {
  mouse.x = event.clientX;
  mouse.y = event.clientY;
};

const handleMouseOut = () => {
  mouse.x = null;
  mouse.y = null;
};

window.addEventListener('mousemove', handleMouseMove);
window.addEventListener('mouseout', handleMouseOut);

init();
animate();

return () => {
  window.removeEventListener('resize', resizeCanvas);
  window.removeEventListener('mousemove', handleMouseMove);
  window.removeEventListener('mouseout', handleMouseOut);
};

```

```

        cancelAnimationFrame(animationFrameId);
    };
}, []);

const fadeUpVariants = {
  hidden: { opacity: 0, y: 20 },
  visible: (i) => ({
    opacity: 1,
    y: 0,
    transition: {
      delay: i * 0.2 + 0.5,
      duration: 0.8,
      ease: "easeInOut",
    },
  }),
};

return (
  // Removed bg-black from this container
  <div className="relative h-screen w-full flex flex-col items-center justify-center
overflow-hidden">
    {/* The canvas is now the primary background */}
    <canvas ref={canvasRef} className="absolute top-0 left-0 w-full h-full"></canvas>

    {/* Overlay HTML Content */}
    <div className="relative z-10 text-center p-6">
      <motion.div
        custom={0}
        variants={fadeUpVariants}
        initial="hidden"
        animate="visible"
        className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full
bg-purple-500/10 border border-purple-500/20 mb-6 backdrop-blur-sm"
      >
        <Zap className="h-4 w-4 text-purple-400" />
        <span className="text-sm font-medium text-gray-200">
          Dynamic Rendering Engine
        </span>
      </motion.div>

      <motion.h1
        custom={1}
        variants={fadeUpVariants}
        initial="hidden"

```

```

        animate="visible"
        className="text-5xl md:text-8xl font-bold tracking-tighter mb-6 bg-clip-text
text-transparent bg-gradient-to-b from-white to-gray-400"
      >
        Aether Flow
      </motion.h1>

      <motion.p
        custom={2}
        variants={fadeUpVariants}
        initial="hidden"
        animate="visible"
        className="max-w-2xl mx-auto text-lg text-gray-400 mb-10"
      >
        An intelligent, adaptive framework for creating fluid digital experiences that feel
        alive and respond to user interaction in real-time.
      </motion.p>

      <motion.div
        custom={3}
        variants={fadeUpVariants}
        initial="hidden"
        animate="visible"
      >
        <button className="px-8 py-4 bg-white text-black font-semibold rounded-lg
shadow-lg hover:bg-gray-200 transition-colors duration-300 flex items-center gap-2 mx-auto">
          Explore the Engine
          <ArrowRight className="h-5 w-5" />
        </button>
      </motion.div>
    </div>
  </div>
);
};

```

```
export default AetherFlowHero;
```

demo.tsx

```
import AetherFlowHero from "@components/ui/aether-flow-hero";
```

```
export default function DemoOne() {
  return <main className="App bg-black">
    <AetherFlowHero />
  </main>
}
```

```
}
```

```
...
```

Install NPM dependencies:

```
```bash  
lucide-react, framer-motion
```
```

Implementation Guidelines

1. Analyze the component structure and identify all required dependencies
2. Review the component's arguments and state
3. Identify any required context providers or hooks and install them
4. Questions to Ask
 - What data/props will be passed to this component?
 - Are there any specific state management requirements?
 - Are there any required assets (images, icons, etc.)?
 - What is the expected responsive behavior?
 - What is the best place to use this component in the app?

Steps to integrate

0. Copy paste all the code above in the correct directories
1. Install external dependencies
2. Fill image assets with Unsplash stock images you know exist
3. Use lucide-react icons for svgs or logos if component requires them

cards

You are given a task to integrate an existing React component in the codebase

The codebase should support:

- shadcn project structure
- Tailwind CSS
- Typescript

If it doesn't, provide instructions on how to setup project via shadcn CLI, install Tailwind or Typescript.

Determine the default path for components and styles.

If default path for components is not /components/ui, provide instructions on why it's important to create this folder

Copy-paste this component to /components/ui folder:

```
```tsx
```

dynamic-frame-layout.tsx

"use client"

import { useState, useEffect, useRef } from "react"

import { motion } from "framer-motion"

interface Frame {

id: number

video: string

defaultPos: { x: number; y: number; w: number; h: number }

corner: string

edgeHorizontal: string

edgeVertical: string

mediaSize: number

borderThickness: number

borderSize: number

isHovered: boolean

}

interface FrameComponentProps {

video: string

width: number | string

height: number | string

className?: string

corner: string

edgeHorizontal: string

edgeVertical: string

mediaSize: number

borderThickness: number

borderSize: number

showFrame: boolean

isHovered: boolean

}

function FrameComponent({

video,

width,

height,

className = "",

corner,

edgeHorizontal,

edgeVertical,

mediaSize,

borderThickness,



```

borderSize,
showFrame,
isHovered,
}: FrameComponentProps) {
 const videoRef = useRef<HTMLVideoElement>(null)

 useEffect(() => {
 if (isHovered) {
 videoRef.current?.play()
 } else {
 videoRef.current?.pause()
 }
 }, [isHovered])

 return (
 <div
 className={`relative ${className}`}
 style={{
 width,
 height,
 transition: "width 0.3s ease-in-out, height 0.3s ease-in-out",
 }}
 >
 <div className="relative w-full h-full overflow-hidden">
 <div
 className="absolute inset-0 flex items-center justify-center"
 style={{
 zIndex: 1,
 transition: "all 0.3s ease-in-out",
 padding: showFrame ? `${borderThickness}px` : "0",
 width: showFrame ? `${borderSize}%` : "100%",
 height: showFrame ? `${borderSize}%` : "100%",
 left: showFrame ? `${(100 - borderSize) / 2}%` : "0",
 top: showFrame ? `${(100 - borderSize) / 2}%` : "0",
 }}
 >
 <div
 className="w-full h-full overflow-hidden"
 style={{
 transform: `scale(${mediaSize})`,
 transformOrigin: "center",
 transition: "transform 0.3s ease-in-out",
 }}
 >

```

```

<video
 className="w-full h-full object-cover"
 src={video}
 loop
 muted
 playsInline
 ref={videoRef}
/>
</div>
</div>

```

```

{showFrame && (
 <div className="absolute inset-0" style={{ zIndex: 2 }}>
 <div
 className="absolute top-0 left-0 w-16 h-16 bg-contain bg-no-repeat"
 style={{ backgroundImage: `url(${corner})` }}
 />
 <div
 className="absolute top-0 right-0 w-16 h-16 bg-contain bg-no-repeat"
 style={{ backgroundImage: `url(${corner})`, transform: "scaleX(-1)" }}
 />
 <div
 className="absolute bottom-0 left-0 w-16 h-16 bg-contain bg-no-repeat"
 style={{ backgroundImage: `url(${corner})`, transform: "scaleY(-1)" }}
 />
 <div
 className="absolute bottom-0 right-0 w-16 h-16 bg-contain bg-no-repeat"
 style={{ backgroundImage: `url(${corner})`, transform: "scale(-1, -1)" }}
 />

 <div
 className="absolute top-0 left-16 right-16 h-16"
 style={{
 backgroundImage: `url(${edgeHorizontal})`,
 backgroundSize: "auto 64px",
 backgroundRepeat: "repeat-x",
 }}
 />
 <div
 className="absolute bottom-0 left-16 right-16 h-16"
 style={{
 backgroundImage: `url(${edgeHorizontal})`,
 backgroundSize: "auto 64px",
 backgroundRepeat: "repeat-x",
 }}
 />
 </div>
)

```

```

 transform: "rotate(180deg)",
 }}
 />
 <div
 className="absolute left-0 top-16 bottom-16 w-16"
 style={{
 backgroundImage: `url(${edgeVertical})`,
 backgroundSize: "64px auto",
 backgroundRepeat: "repeat-y",
 }}
 />
 <div
 className="absolute right-0 top-16 bottom-16 w-16"
 style={{
 backgroundImage: `url(${edgeVertical})`,
 backgroundSize: "64px auto",
 backgroundRepeat: "repeat-y",
 transform: "scaleX(-1)",
 }}
 />
 </div>
)}
</div>
</div>
)
}

```

```

interface DynamicFrameLayoutProps {
 frames: Frame[]
 className?: string
 showFrames?: boolean
 hoverSize?: number
 gapSize?: number
}

```

```

export function DynamicFrameLayout({
 frames: initialFrames,
 className,
 showFrames = false,
 hoverSize = 6,
 gapSize = 4
}: DynamicFrameLayoutProps) {
 const [frames] = useState<Frame[]>(initialFrames)
 const [hovered, setHovered] = useState<{ row: number; col: number } | null>(null)

```

```

const getRowSizes = () => {
 if (hovered === null) return "4fr 4fr 4fr"
 const { row } = hovered
 const nonHoveredSize = (12 - hoverSize) / 2
 return [0, 1, 2].map((r) => (r === row ? `${hoverSize}fr` : `${nonHoveredSize}fr`)).join(" ")
}

```

```

const getColSizes = () => {
 if (hovered === null) return "4fr 4fr 4fr"
 const { col } = hovered
 const nonHoveredSize = (12 - hoverSize) / 2
 return [0, 1, 2].map((c) => (c === col ? `${hoverSize}fr` : `${nonHoveredSize}fr`)).join(" ")
}

```

```

const getTransformOrigin = (x: number, y: number) => {
 const vertical = y === 0 ? "top" : y === 4 ? "center" : "bottom"
 const horizontal = x === 0 ? "left" : x === 4 ? "center" : "right"
 return `${vertical} ${horizontal}`
}

```

```

return (
 <div
 className={`relative w-full h-full ${className}`}
 style={{
 display: "grid",
 gridTemplateRows: getRowSizes(),
 gridTemplateColumns: getColSizes(),
 gap: `${gapSize}px`,
 transition: "grid-template-rows 0.4s ease, grid-template-columns 0.4s ease",
 }}
 >
 {frames.map((frame) => {
 const row = Math.floor(frame.defaultPos.y / 4)
 const col = Math.floor(frame.defaultPos.x / 4)
 const transformOrigin = getTransformOrigin(frame.defaultPos.x, frame.defaultPos.y)

 return (
 <motion.div
 key={frame.id}
 className="relative"
 style={{
 transformOrigin,
 transition: "transform 0.4s ease",

```

```

 }}
 onMouseEnter={() => setHovered({ row, col })}
 onMouseLeave={() => setHovered(null)}
 >
 <FrameComponent
 video={frame.video}
 width="100%"
 height="100%"
 className="absolute inset-0"
 corner={frame.corner}
 edgeHorizontal={frame.edgeHorizontal}
 edgeVertical={frame.edgeVertical}
 mediaSize={frame.mediaSize}
 borderThickness={frame.borderThickness}
 borderSize={frame.borderSize}
 showFrame={showFrames}
 isHovered={hovered?.row === row && hovered?.col === col}
 />
 </motion.div>
)
}}
</div>
)
}

```

demo.tsx  
"use client"

```

import { DynamicFrameLayout } from "@components/ui/dynamic-frame-layout"

const demoFrames = [
 {
 id: 1,
 video:
 "https://static.cdn-luma.com/files/981e483f71aa764b/Company%20Thing%20Exported.mp4",
 defaultPos: { x: 0, y: 0, w: 4, h: 4 },
 mediaSize: 1,
 isHovered: false,
 },
 {
 id: 2,
 video:
 "https://static.cdn-luma.com/files/58ab7363888153e3/WebGL%20Exported%20(1).mp4",
 defaultPos: { x: 4, y: 0, w: 4, h: 4 },
 },
]

```

```
 mediaSize: 1,
 isHovered: false,
 },
 {
 id: 3,
 video:
"https://static.cdn-luma.com/files/58ab7363888153e3/Jitter%20Exported%20Poster.mp4",
 defaultPos: { x: 8, y: 0, w: 4, h: 4 },
 mediaSize: 1,
 isHovered: false,
 },
 {
 id: 4,
 video:
"https://static.cdn-luma.com/files/58ab7363888153e3/Exported%20Web%20Video.mp4",
 defaultPos: { x: 0, y: 4, w: 4, h: 4 },
 mediaSize: 1,
 isHovered: false,
 },
 {
 id: 5,
 video: "https://static.cdn-luma.com/files/58ab7363888153e3/Logo%20Exported.mp4",
 defaultPos: { x: 4, y: 4, w: 4, h: 4 },
 mediaSize: 1,
 isHovered: false,
 },
 {
 id: 6,
 video:
"https://static.cdn-luma.com/files/58ab7363888153e3/Animation%20Exported%20(4).mp4",
 defaultPos: { x: 8, y: 4, w: 4, h: 4 },
 mediaSize: 1,
 isHovered: false,
 },
 {
 id: 7,
 video:
"https://static.cdn-luma.com/files/58ab7363888153e3/Illustration%20Exported%20(1).mp4",
 defaultPos: { x: 0, y: 8, w: 4, h: 4 },
 mediaSize: 1,
 isHovered: false,
 },
 {
 id: 8,
```

```

 video:
 "https://static.cdn-luma.com/files/58ab7363888153e3/Art%20Direction%20Exported.mp4",
 defaultPos: { x: 4, y: 8, w: 4, h: 4 },
 mediaSize: 1,
 isHovered: false,
 },
 {
 id: 9,
 video: "https://static.cdn-luma.com/files/58ab7363888153e3/Product%20Video.mp4",
 defaultPos: { x: 8, y: 8, w: 4, h: 4 },
 mediaSize: 1,
 isHovered: false,
 },
]

```

```

export function DemoPage() {
 return (
 <div className="h-screen w-screen bg-zinc-900">
 <DynamicFrameLayout
 frames={demoFrames}
 className="w-full h-full"
 hoverSize={6}
 gapSize={4}
 />
 </div>
)
}
...

```

Install NPM dependencies:

```

```bash
framer-motion
```

```

## Implementation Guidelines

1. Analyze the component structure and identify all required dependencies
2. Review the component's arguments and state
3. Identify any required context providers or hooks and install them
4. Questions to Ask
  - What data/props will be passed to this component?
  - Are there any specific state management requirements?
  - Are there any required assets (images, icons, etc.)?
  - What is the expected responsive behavior?
  - What is the best place to use this component in the app?

Steps to integrate

0. Copy paste all the code above in the correct directories
1. Install external dependencies
2. Fill image assets with Unsplash stock images you know exist
3. Use lucide-react icons for svgs or logos if component requires them

## Scroll: Sticky scroll

You are given a task to integrate an existing React component in the codebase

The codebase should support:

- shadcn project structure
- Tailwind CSS
- Typescript

If it doesn't, provide instructions on how to setup project via shadcn CLI, install Tailwind or Typescript.

Determine the default path for components and styles.

If default path for components is not /components/ui, provide instructions on why it's important to create this folder

Copy-paste this component to /components/ui folder:

```
```.tsx
sticky-scroll-reveal.tsx
"use client";
import React, { useEffect, useRef, useState } from "react";
import { useMotionValueEvent, useScroll } from "framer-motion";
import { motion } from "framer-motion";
import { cn } from "@/lib/utils";

export const StickyScroll = ({
  content,
  contentClassName,
}: {
  content: {
    title: string;
    description: string;
    content?: React.ReactNode | any;
  }[];
  contentClassName?: string;
}) => {
```



```
const [activeCard, setActiveCard] = React.useState(0);
```

```
const ref = useRef<any>(null);
```

```
const { scrollYProgress } = useScroll({
```

```
  container: ref,
```

```
  offset: ["start start", "end start"],
```

```
});
```

```
const cardLength = content.length;
```

```
useMotionValueEvent(scrollYProgress, "change", (latest) => {
```

```
  const cardsBreakpoints = content.map( (_, index) => index / cardLength);
```

```
  const closestBreakpointIndex = cardsBreakpoints.reduce(
```

```
    (acc, breakpoint, index) => {
```

```
      const distance = Math.abs(latest - breakpoint);
```

```
      if (distance < Math.abs(latest - cardsBreakpoints[acc])) {
```

```
        return index;
```

```
      }
```

```
      return acc;
```

```
    },
```

```
    0
```

```
  );
```

```
  setActiveCard(closestBreakpointIndex);
```

```
});
```

```
const backgroundColors = [
```

```
  "rgb(15 23 42)", // slate-900
```

```
  "rgb(0 0 0)", // black
```

```
  "rgb(23 23 23)", // neutral-900
```

```
];
```

```
const linearGradients = [
```

```
  "linear-gradient(to bottom right, rgb(6 182 212), rgb(16 185 129))", // cyan-500 to emerald-500
```

```
  "linear-gradient(to bottom right, rgb(236 72 153), rgb(99 102 241))", // pink-500 to indigo-500
```

```
  "linear-gradient(to bottom right, rgb(249 115 22), rgb(234 179 8))", // orange-500 to
```

```
yellow-500
```

```
];
```

```
const [backgroundGradient, setBackgroundGradient] = useState(
```

```
  linearGradients[0]
```

```
);
```

```
useEffect(() => {
```

```
  setBackgroundGradient(linearGradients[activeCard % linearGradients.length]);
```

```
}, [activeCard]);
```

```

return (
  <motion.div
    animate={{
      backgroundColor: backgroundColors[activeCard % backgroundColors.length],
    }}
    className="h-[30rem] overflow-y-auto flex justify-center relative space-x-10 rounded-md
p-10"
    ref={ref}
  >
    <div className="div relative flex items-start px-4">
      <div className="max-w-2xl">
        {content.map((item, index) => (
          <div key={item.title + index} className="my-20">
            <motion.h2
              initial={{
                opacity: 0,
              }}
              animate={{
                opacity: activeCard === index ? 1 : 0.3,
              }}
              className="text-2xl font-bold text-slate-100"
            >
              {item.title}
            </motion.h2>
            <motion.p
              initial={{
                opacity: 0,
              }}
              animate={{
                opacity: activeCard === index ? 1 : 0.3,
              }}
              className="text-kg text-slate-300 max-w-sm mt-10"
            >
              {item.description}
            </motion.p>
          </div>
        ))}
      <div className="h-40" />
    </div>
  </div>
  <div
    style={{ background: backgroundGradient }}
    className={cn(
      "hidden lg:block h-60 w-80 rounded-md bg-white sticky top-10 overflow-hidden",

```

```

        contentClassName
      )}
    >
    {content[activeCard].content ?? null}
  </div>
</motion.div>
);
};

```

```

demo.tsx
"use client";
import React from "react";
import { StickyScroll } from "@components/ui/sticky-scroll-reveal";
import Image from "next/image";

```

```

const content = [
  {
    title: "Collaborative Editing",
    description:
      "Work together in real time with your team, clients, and stakeholders. Collaborate on documents, share ideas, and make decisions quickly. With our platform, you can streamline your workflow and increase productivity.",
    content: (
      <div className="h-full w-full
bg-[linear-gradient(to_bottom_right,var(--cyan-500),var(--emerald-500))] flex items-center
justify-center text-white">
        Collaborative Editing
      </div>
    ),
  },
  {
    title: "Real time changes",
    description:
      "See changes as they happen. With our platform, you can track every modification in real time. No more confusion about the latest version of your project. Say goodbye to the chaos of version control and embrace the simplicity of real-time updates.",
    content: (
      <div className="h-full w-full flex items-center justify-center text-white">
        <Image
          src="https://ui.aceternity.com/_next/image?url=%2Flinear.webp&w=640&q=75"
          width={300}
          height={300}
          className="h-full w-full object-cover"
          alt="linear board demo"
        />
      </div>
    ),
  },
];

```

```

    />
  </div>
),
},
{
  title: "Version control",
  description:
    "Experience real-time updates and never stress about version control again. Our platform ensures that you're always working on the most recent version of your project, eliminating the need for constant manual updates. Stay in the loop, keep your team aligned, and maintain the flow of your work without any interruptions.",
  content: (
    <div className="h-full w-full
bg-[linear-gradient(to_bottom_right,var(--orange-500),var(--yellow-500))] flex items-center justify-center text-white">
      Version control
    </div>
  ),
},
{
  title: "Running out of content",
  description:
    "Experience real-time updates and never stress about version control again. Our platform ensures that you're always working on the most recent version of your project, eliminating the need for constant manual updates. Stay in the loop, keep your team aligned, and maintain the flow of your work without any interruptions.",
  content: (
    <div className="h-full w-full
bg-[linear-gradient(to_bottom_right,var(--cyan-500),var(--emerald-500))] flex items-center justify-center text-white">
      Running out of content
    </div>
  ),
},
];

export function StickyScrollRevealDemo() {
  return (
    <div className="p-10">
      <StickyScroll content={content} />
    </div>
  );
}

```

```

Install NPM dependencies:

```
```bash
framer-motion
```
```

#### Implementation Guidelines

1. Analyze the component structure and identify all required dependencies
2. Review the component's arguments and state
3. Identify any required context providers or hooks and install them
4. Questions to Ask
  - What data/props will be passed to this component?
  - Are there any specific state management requirements?
  - Are there any required assets (images, icons, etc.)?
  - What is the expected responsive behavior?
  - What is the best place to use this component in the app?

#### Steps to integrate

0. Copy paste all the code above in the correct directories
1. Install external dependencies
2. Fill image assets with Unsplash stock images you know exist
3. Use lucide-react icons for svgs or logos if component requires them

## Scroll : effect two

You are given a task to integrate an existing React component in the codebase

The codebase should support:

- shadcn project structure
- Tailwind CSS
- Typescript

If it doesn't, provide instructions on how to setup project via shadcn CLI, install Tailwind or Typescript.

Determine the default path for components and styles.

If default path for components is not /components/ui, provide instructions on why it's important to create this folder

Copy-paste this component to /components/ui folder:

```
```tsx
```

container-scroll-animation.tsx

```
"use client";
import React, { useRef } from "react";
import { useScroll, useTransform, motion, MotionValue } from "framer-motion";

export const ContainerScroll = ({
  titleComponent,
  children,
}: {
  titleComponent: string | React.ReactNode;
  children: React.ReactNode;
}) => {
  const containerRef = useRef<HTMLDivElement>(null);
  const { scrollYProgress } = useScroll({
    target: containerRef,
  });
  const [isMobile, setIsMobile] = React.useState(false);

  React.useEffect(() => {
    const checkMobile = () => {
      setIsMobile(window.innerWidth <= 768);
    };
    checkMobile();
    window.addEventListener("resize", checkMobile);
    return () => {
      window.removeEventListener("resize", checkMobile);
    };
  }, []);

  const scaleDimensions = () => {
    return isMobile ? [0.7, 0.9] : [1.05, 1];
  };

  const rotate = useTransform(scrollYProgress, [0, 1], [20, 0]);
  const scale = useTransform(scrollYProgress, [0, 1], scaleDimensions());
  const translate = useTransform(scrollYProgress, [0, 1], [0, -100]);

  return (
    <div
      className="h-[60rem] md:h-[80rem] flex items-center justify-center relative p-2 md:p-20"
      ref={containerRef}
    >
      <div
        className="py-10 md:py-40 w-full relative"

```

```

        style={{
          perspective: "1000px",
        }}
      >
        <Header translate={translate} titleComponent={titleComponent} />
        <Card rotate={rotate} translate={translate} scale={scale}>
          {children}
        </Card>
      </div>
    </div>
  );
};

```

```

export const Header = ({ translate, titleComponent }: any) => {
  return (
    <motion.div
      style={{
        translateY: translate,
      }}
      className="div max-w-5xl mx-auto text-center"
    >
      {titleComponent}
    </motion.div>
  );
};

```

```

export const Card = ({
  rotate,
  scale,
  children,
}: {
  rotate: MotionValue<number>;
  scale: MotionValue<number>;
  translate: MotionValue<number>;
  children: React.ReactNode;
}) => {
  return (
    <motion.div
      style={{
        rotateX: rotate,
        scale,
        boxShadow:
          "0 0 #0000004d, 0 9px 20px #0000004a, 0 37px 37px #00000042, 0 84px 50px
          #00000026, 0 149px 60px #0000000a, 0 233px 65px #00000003",

```

```

    }}
    className="max-w-5xl -mt-12 mx-auto h-[30rem] md:h-[40rem] w-full border-4
border-[#6C6C6C] p-2 md:p-6 bg-[#222222] rounded-[30px] shadow-2xl"
  >
    <div className=" h-full w-full overflow-hidden rounded-2xl bg-gray-100 dark:bg-zinc-900
md:rounded-2xl md:p-4 ">
      {children}
    </div>
  </motion.div>
);
};

```

demo.tsx

```

"use client";
import React from "react";
import { ContainerScroll } from "@components/ui/container-scroll-animation";
import Image from "next/image";

export function HeroScrollDemo() {
  return (
    <div className="flex flex-col overflow-hidden pb-[500px] pt-[1000px]">
      <ContainerScroll
        titleComponent={
          <>
            <h1 className="text-4xl font-semibold text-black dark:text-white">
              Unleash the power of <br />
              <span className="text-4xl md:text-[6rem] font-bold mt-1 leading-none">
                Scroll Animations
              </span>
            </h1>
          </>
        }
      >
        <Image
          src={`https://ui.aceternity.com/_next/image?url=%2Flinear.webp&w=3840&q=75`}
          alt="hero"
          height={720}
          width={1400}
          className="mx-auto rounded-2xl object-cover h-full object-left-top"
          draggable={false}
        />
      </ContainerScroll>
    </div>
  );
}

```



```
);  
}  
...  

```

Install NPM dependencies:

```
```bash  
framer-motion
...

```

### Implementation Guidelines

1. Analyze the component structure and identify all required dependencies
2. Review the component's arguments and state
3. Identify any required context providers or hooks and install them
4. Questions to Ask
  - What data/props will be passed to this component?
  - Are there any specific state management requirements?
  - Are there any required assets (images, icons, etc.)?
  - What is the expected responsive behavior?
  - What is the best place to use this component in the app?

### Steps to integrate

0. Copy paste all the code above in the correct directories
1. Install external dependencies
2. Fill image assets with Unsplash stock images you know exist
3. Use lucide-react icons for svgs or logos if component requires them

## Scroll effect 3

You are given a task to integrate an existing React component in the codebase

The codebase should support:

- shadcn project structure
- Tailwind CSS
- Typescript

If it doesn't, provide instructions on how to setup project via shadcn CLI, install Tailwind or Typescript.

Determine the default path for components and styles.

If default path for components is not /components/ui, provide instructions on why it's important to create this folder

Copy-paste this component to /components/ui folder:

```
```:tsx
scroll-expansion-hero.tsx
'use client';

import {
  useEffect,
  useRef,
  useState,
  ReactNode,
  TouchEvent,
  WheelEvent,
} from 'react';
import Image from 'next/image';
import { motion } from 'framer-motion';

interface ScrollExpandMediaProps {
  mediaType?: 'video' | 'image';
  mediaSrc: string;
  posterSrc?: string;
  bgImageSrc: string;
  title?: string;
  date?: string;
  scrollToExpand?: string;
  textBlend?: boolean;
  children?: ReactNode;
}

const ScrollExpandMedia = ({
  mediaType = 'video',
  mediaSrc,
  posterSrc,
  bgImageSrc,
  title,
  date,
  scrollToExpand,
  textBlend,
  children,
}: ScrollExpandMediaProps) => {
  const [scrollProgress, setScrollProgress] = useState<number>(0);
  const [showContent, setShowContent] = useState<boolean>(false);
  const [mediaFullyExpanded, setMediaFullyExpanded] = useState<boolean>(false);
  const [touchStartY, setTouchStartY] = useState<number>(0);
  const [isMobileState, setIsMobileState] = useState<boolean>(false);
```

```
const sectionRef = useRef<HTMLDivElement | null>(null);
```

```
useEffect(() => {  
  setScrollProgress(0);  
  setShowContent(false);  
  setMediaFullyExpanded(false);  
}, [mediaType]);
```

```
useEffect(() => {  
  const handleWheel = (e: WheelEvent) => {  
    if (mediaFullyExpanded && e.deltaY < 0 && window.scrollY <= 5) {  
      setMediaFullyExpanded(false);  
      e.preventDefault();  
    } else if (!mediaFullyExpanded) {  
      e.preventDefault();  
      const scrollDelta = e.deltaY * 0.0009;  
      const newProgress = Math.min(  
        Math.max(scrollProgress + scrollDelta, 0),  
        1  
      );  
      setScrollProgress(newProgress);  
  
      if (newProgress >= 1) {  
        setMediaFullyExpanded(true);  
        setShowContent(true);  
      } else if (newProgress < 0.75) {  
        setShowContent(false);  
      }  
    }  
  };  
};
```

```
const handleTouchStart = (e: TouchEvent) => {  
  setTouchStartY(e.touches[0].clientY);  
};
```

```
const handleTouchMove = (e: TouchEvent) => {  
  if (!touchStartY) return;  
  
  const touchY = e.touches[0].clientY;  
  const deltaY = touchStartY - touchY;  
  
  if (mediaFullyExpanded && deltaY < -20 && window.scrollY <= 5) {  
    setMediaFullyExpanded(false);  
    e.preventDefault();  
  }  
};
```

```

} else if (!mediaFullyExpanded) {
  e.preventDefault();
  // Increase sensitivity for mobile, especially when scrolling back
  const scrollFactor = deltaY < 0 ? 0.008 : 0.005; // Higher sensitivity for scrolling back
  const scrollDelta = deltaY * scrollFactor;
  const newProgress = Math.min(
    Math.max(scrollProgress + scrollDelta, 0),
    1
  );
  setScrollProgress(newProgress);

  if (newProgress >= 1) {
    setMediaFullyExpanded(true);
    setShowContent(true);
  } else if (newProgress < 0.75) {
    setShowContent(false);
  }

  setTouchStartY(touchY);
}
};

const handleTouchEnd = (): void => {
  setTouchStartY(0);
};

const handleScroll = (): void => {
  if (!mediaFullyExpanded) {
    window.scrollTo(0, 0);
  }
};

window.addEventListener('wheel', handleWheel as unknown as EventListener, {
  passive: false,
});
window.addEventListener('scroll', handleScroll as EventListener);
window.addEventListener(
  'touchstart',
  handleTouchStart as unknown as EventListener,
  { passive: false }
);
window.addEventListener(
  'touchmove',
  handleTouchMove as unknown as EventListener,

```

```

    { passive: false }
  );
  window.addEventListener('touchend', handleTouchEnd as EventListener);

  return () => {
    window.removeEventListener(
      'wheel',
      handleWheel as unknown as EventListener
    );
    window.removeEventListener('scroll', handleScroll as EventListener);
    window.removeEventListener(
      'touchstart',
      handleTouchStart as unknown as EventListener
    );
    window.removeEventListener(
      'touchmove',
      handleTouchMove as unknown as EventListener
    );
    window.removeEventListener('touchend', handleTouchEnd as EventListener);
  };
}, [scrollProgress, mediaFullyExpanded, touchStartY]);

useEffect(() => {
  const checkIfMobile = (): void => {
    setIsMobileState(window.innerWidth < 768);
  };

  checkIfMobile();
  window.addEventListener('resize', checkIfMobile);

  return () => window.removeEventListener('resize', checkIfMobile);
}, []);

const mediaWidth = 300 + scrollProgress * (isMobileState ? 650 : 1250);
const mediaHeight = 400 + scrollProgress * (isMobileState ? 200 : 400);
const textTranslateX = scrollProgress * (isMobileState ? 180 : 150);

const firstWord = title ? title.split(' ')[0] : "";
const restOfTitle = title ? title.split(' ').slice(1).join(' ') : "";

return (
  <div
    ref={sectionRef}
    className='transition-colors duration-700 ease-in-out overflow-x-hidden'

```

```

>
<section className='relative flex flex-col items-center justify-start min-h-[100dvh]'>
  <div className='relative w-full flex flex-col items-center min-h-[100dvh]'>
    <motion.div
      className='absolute inset-0 z-0 h-full'
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 - scrollProgress }}
      transition={{ duration: 0.1 }}
    >
      <Image
        src={bgImageSrc}
        alt='Background'
        width={1920}
        height={1080}
        className='w-screen h-screen'
        style={{
          objectFit: 'cover',
          objectPosition: 'center',
        }}
        priority
      />
      <div className='absolute inset-0 bg-black/10' />
    </motion.div>

    <div className='container mx-auto flex flex-col items-center justify-start relative z-10'>
      <div className='flex flex-col items-center justify-center w-full h-[100dvh] relative'>
        <div
          className='absolute z-0 top-1/2 left-1/2 transform -translate-x-1/2 -translate-y-1/2
transition-none rounded-2xl'
          style={{
            width: `${mediaWidth}px`,
            height: `${mediaHeight}px`,
            maxWidth: '95vw',
            maxHeight: '85vh',
            boxShadow: '0px 0px 50px rgba(0, 0, 0, 0.3)',
          }}
        >
          {mediaType === 'video' ? (
            mediaSrc.includes('youtube.com') ? (
              <div className='relative w-full h-full pointer-events-none'>
                <iframe
                  width='100%'
                  height='100%'
                  src={

```

```

mediaSrc.includes('embed')
? mediaSrc +
(mediaSrc.includes('?') ? '&' : '?') +

'autoplay=1&mute=1&loop=1&controls=0&showinfo=0&rel=0&disablekb=1&modestbranding=1'
: mediaSrc.replace('watch?v=', 'embed/') +

'?autoplay=1&mute=1&loop=1&controls=0&showinfo=0&rel=0&disablekb=1&modestbranding=1
&playlist=' +
mediaSrc.split('v=')[1]
}
className='w-full h-full rounded-xl'
frameBorder='0'
allow='accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
picture-in-picture'
allowFullScreen
/>
<div
className='absolute inset-0 z-10'
style={{ pointerEvents: 'none' }}
></div>

<motion.div
className='absolute inset-0 bg-black/30 rounded-xl'
initial={{ opacity: 0.7 }}
animate={{ opacity: 0.5 - scrollProgress * 0.3 }}
transition={{ duration: 0.2 }}
/>
</div>
): (
<div className='relative w-full h-full pointer-events-none'>
<video
src={mediaSrc}
poster={posterSrc}
autoPlay
muted
loop
playsInline
preload='auto'
className='w-full h-full object-cover rounded-xl'
controls={false}
disablePictureInPicture
disableRemotePlayback
/>

```

```

    <div
      className='absolute inset-0 z-10'
      style={{ pointerEvents: 'none' }}
    ></div>

    <motion.div
      className='absolute inset-0 bg-black/30 rounded-xl'
      initial={{ opacity: 0.7 }}
      animate={{ opacity: 0.5 - scrollProgress * 0.3 }}
      transition={{ duration: 0.2 }}
    />
  </div>
)
): (
  <div className='relative w-full h-full'>
    <Image
      src={mediaSrc}
      alt={title || 'Media content'}
      width={1280}
      height={720}
      className='w-full h-full object-cover rounded-xl'
    />

    <motion.div
      className='absolute inset-0 bg-black/50 rounded-xl'
      initial={{ opacity: 0.7 }}
      animate={{ opacity: 0.7 - scrollProgress * 0.3 }}
      transition={{ duration: 0.2 }}
    />
  </div>
)}

  <div className='flex flex-col items-center text-center relative z-10 mt-4
transition-none'>
    {date && (
      <p
        className='text-2xl text-blue-200'
        style={{ transform: `translateX(-${textTranslateX}vw)` }}
      >
        {date}
      </p>
    )}
    {scrollToExpand && (
      <p

```



```

        className='text-blue-200 font-medium text-center'
        style={{ transform: `translateX(${textTranslateX}vw)` }}
      >
        {scrollToExpand}
      </p>
    )}
  </div>
</div>

<div
  className={`flex items-center justify-center text-center gap-4 w-full relative z-10
  transition-none flex-col ${
    textBlend ? 'mix-blend-difference' : 'mix-blend-normal'
  }}`
  >
    <motion.h2
      className='text-4xl md:text-5xl lg:text-6xl font-bold text-blue-200 transition-none'
      style={{ transform: `translateX(-${textTranslateX}vw)` }}
    >
      {firstWord}
    </motion.h2>
    <motion.h2
      className='text-4xl md:text-5xl lg:text-6xl font-bold text-center text-blue-200
  transition-none'
      style={{ transform: `translateX(${textTranslateX}vw)` }}
    >
      {restOfTitle}
    </motion.h2>
  </div>
</div>

<motion.section
  className='flex flex-col w-full px-8 py-10 md:px-16 lg:py-20'
  initial={{ opacity: 0 }}
  animate={{ opacity: showContent ? 1 : 0 }}
  transition={{ duration: 0.7 }}
  >
    {children}
  </motion.section>
</div>
</div>
</section>
</div>
);

```

```
};
```

```
export default ScrollExpandMedia;
```

```
demo.tsx
```

```
'use client';
```

```
import { useState, useEffect } from 'react';
```

```
import ScrollExpandMedia from '@components/blocks/scroll-expansion-hero';
```

```
interface MediaAbout {  
  overview: string;  
  conclusion: string;  
}
```

```
interface MediaContent {  
  src: string;  
  poster?: string;  
  background: string;  
  title: string;  
  date: string;  
  scrollToExpand: string;  
  about: MediaAbout;  
}
```

```
interface MediaContentCollection {  
  [key: string]: MediaContent;  
}
```

```
const sampleMediaContent: MediaContentCollection = {  
  video: {  
    src:  
'https://me7aitdbxq.ufs.sh/f/2wsMIGDMQRdYuZ5R8ahEEZ4aQK56LizRdfBSqeDMsmUIrJN1',  
    poster:  
      'https://images.pexels.com/videos/5752729/space-earth-universe-cosmos-5752729.jpeg',  
    background:  
  
'https://me7aitdbxq.ufs.sh/f/2wsMIGDMQRdYMNjMIBUYHaeYpxduXPVNwf8mnFA61L7rkcoS',  
    title: 'Immersive Video Experience',  
    date: 'Cosmic Journey',  
    scrollToExpand: 'Scroll to Expand Demo',  
    about: {  
      overview:
```

'This is a demonstration of the ScrollExpandMedia component with a video. As you scroll, the video expands to fill more of the screen, creating an immersive experience. This component is perfect for showcasing video content in a modern, interactive way.',

conclusion:

'The ScrollExpandMedia component provides a unique way to engage users with your content through interactive scrolling. Try switching between video and image modes to see different implementations.',

```
    },  
    },  
    image: {  
      src:  
'https://images.unsplash.com/photo-1682687982501-1e58ab814714?q=80&w=1280&auto=format&fit=crop',  
      background:
```

```
'https://images.unsplash.com/photo-1506744038136-46273834b3fb?q=80&w=1920&auto=format&fit=crop',
```

```
    title: 'Dynamic Image Showcase',  
    date: 'Underwater Adventure',  
    scrollToExpand: 'Scroll to Expand Demo',  
    about: {  
      overview:
```

'This is a demonstration of the ScrollExpandMedia component with an image. The same smooth expansion effect works beautifully with static images, allowing you to create engaging visual experiences without video content.',

conclusion:

'The ScrollExpandMedia component works equally well with images and videos. This flexibility allows you to choose the media type that best suits your content while maintaining the same engaging user experience.',

```
    },  
    },  
  };
```

```
const MediaContent = ({ mediaType }: { mediaType: 'video' | 'image' }) => {  
  const currentMedia = sampleMediaContent[mediaType];
```

```
  return (  
    <div className='max-w-4xl mx-auto'>  
      <h2 className='text-3xl font-bold mb-6 text-black dark:text-white'>  
        About This Component  
      </h2>  
      <p className='text-lg mb-8 text-black dark:text-white'>  
        {currentMedia.about.overview}  
      </p>
```

```

    <p className='text-lg mb-8 text-black dark:text-white'>
      {currentMedia.about.conclusion}
    </p>
  </div>
);
};

```

```

export const VideoExpansionTextBlend = () => {
  const mediaType = 'video';
  const currentMedia = sampleMediaContent[mediaType];

```

```

  useEffect(() => {
    window.scrollTo(0, 0);

```

```

    const resetEvent = new Event('resetSection');
    window.dispatchEvent(resetEvent);
  }, []);

```

```

return (
  <div className='min-h-screen'>
    <ScrollExpandMedia
      mediaType={mediaType}
      mediaSrc={currentMedia.src}
      posterSrc={currentMedia.poster}
      bgImageSrc={currentMedia.background}
      title={currentMedia.title}
      date={currentMedia.date}
      scrollToExpand={currentMedia.scrollToExpand}
      textBlend
    >
      <MediaContent mediaType={mediaType} />
    </ScrollExpandMedia>
  </div>
);
};

```

```

export const ImageExpansionTextBlend = () => {
  const mediaType = 'image';
  const currentMedia = sampleMediaContent[mediaType];

```

```

  useEffect(() => {
    window.scrollTo(0, 0);

```

```

    const resetEvent = new Event('resetSection');
    window.dispatchEvent(resetEvent);
  }, []);

  return (
    <div className='min-h-screen'>
      <ScrollExpandMedia
        mediaType={mediaType}
        mediaSrc={currentMedia.src}
        bgImageSrc={currentMedia.background}
        title={currentMedia.title}
        date={currentMedia.date}
        scrollToExpand={currentMedia.scrollToExpand}
        textBlend
      >
        <MediaContent mediaType={mediaType} />
      </ScrollExpandMedia>
    </div>
  );
};

```

```

export const VideoExpansion = () => {
  const mediaType = 'video';
  const currentMedia = sampleMediaContent[mediaType];

```

```

  useEffect(() => {
    window.scrollTo(0, 0);

```

```

    const resetEvent = new Event('resetSection');
    window.dispatchEvent(resetEvent);
  }, []);

```

```

  return (
    <div className='min-h-screen'>
      <ScrollExpandMedia
        mediaType={mediaType}
        mediaSrc={currentMedia.src}
        posterSrc={currentMedia.poster}
        bgImageSrc={currentMedia.background}
        title={currentMedia.title}
        date={currentMedia.date}
        scrollToExpand={currentMedia.scrollToExpand}
      >
        <MediaContent mediaType={mediaType} />

```

```

        </ScrollExpandMedia>
    </div>
);
};

export const ImageExpansion = () => {
    const mediaType = 'image';
    const currentMedia = sampleMediaContent[mediaType];

    useEffect(() => {
        window.scrollTo(0, 0);

        const resetEvent = new Event('resetSection');
        window.dispatchEvent(resetEvent);
    }, []);

    return (
        <div className='min-h-screen'>
            <ScrollExpandMedia
                mediaType={mediaType}
                mediaSrc={currentMedia.src}
                bgImageSrc={currentMedia.background}
                title={currentMedia.title}
                date={currentMedia.date}
                scrollToExpand={currentMedia.scrollToExpand}
            >
                <MediaContent mediaType={mediaType} />
            </ScrollExpandMedia>
        </div>
    );
};

const Demo = () => {
    const [mediaType, setMediaType] = useState('video');
    const currentMedia = sampleMediaContent[mediaType];

    useEffect(() => {
        window.scrollTo(0, 0);

        const resetEvent = new Event('resetSection');
        window.dispatchEvent(resetEvent);
    }, [mediaType]);

    return (

```

```

<div className='min-h-screen'>
  <div className='fixed top-4 right-4 z-50 flex gap-2'>
    <button
      onClick={() => setMediaType('video')}
      className={`px-4 py-2 rounded-lg ${
        mediaType === 'video'
          ? 'bg-white text-black'
          : 'bg-black/50 text-white border border-white/30'
      }}
    >
      Video
    </button>

    <button
      onClick={() => setMediaType('image')}
      className={`px-4 py-2 rounded-lg ${
        mediaType === 'image'
          ? 'bg-white text-black'
          : 'bg-black/50 text-white border border-white/30'
      }}
    >
      Image
    </button>
  </div>

  <ScrollExpandMedia
    mediaType={mediaType as 'video' | 'image'}
    mediaSrc={currentMedia.src}
    posterSrc={mediaType === 'video' ? currentMedia.poster : undefined}
    bgImageSrc={currentMedia.background}
    title={currentMedia.title}
    date={currentMedia.date}
    scrollToExpand={currentMedia.scrollToExpand}
  >
    <MediaContent mediaType={mediaType as 'video' | 'image'} />
  </ScrollExpandMedia>
</div>
);
};

export default Demo;

...

```

Install NPM dependencies:

```
```bash
framer-motion
```
```

Implementation Guidelines

1. Analyze the component structure and identify all required dependencies
2. Review the component's arguments and state
3. Identify any required context providers or hooks and install them
4. Questions to Ask
 - What data/props will be passed to this component?
 - Are there any specific state management requirements?
 - Are there any required assets (images, icons, etc.)?
 - What is the expected responsive behavior?
 - What is the best place to use this component in the app?

Steps to integrate

0. Copy paste all the code above in the correct directories
1. Install external dependencies
2. Fill image assets with Unsplash stock images you know exist
3. Use lucide-react icons for svgs or logos if component requires them

Animated card gravity boxes

You are given a task to integrate an existing React component in the codebase

The codebase should support:

- shadcn project structure
- Tailwind CSS
- Typescript

If it doesn't, provide instructions on how to setup project via shadcn CLI, install Tailwind or Typescript.

Determine the default path for components and styles.

If default path for components is not /components/ui, provide instructions on why it's important to create this folder

Copy-paste this component to /components/ui folder:

```
```tsx
gravity.tsx
import {
```



```

 createContext,
 forwardRef,
 ReactNode,
 useCallback,
 useContext,
 useEffect,
 useImperativeHandle,
 useRef,
 useState,
 } from "react"
import { debounce } from "lodash"
import Matter, {
 Bodies,
 Common,
 Engine,
 Events,
 Mouse,
 MouseConstraint,
 Query,
 Render,
 Runner,
 World,
} from "matter-js"

import { cn } from "@lib/utils"

import SVGPathCommander from 'svg-path-commander';

// Function to convert SVG path "d" to vertices
function parsePathToVertices(path: string, sampleLength = 15) {
 // Convert path to absolute commands
 const commander = new SVGPathCommander(path);

 const points: { x: number, y: number }[] = [];
 let lastPoint: { x: number, y: number } | null = null;

 // Get total length of the path
 const totalLength = commander.getTotalLength();
 let length = 0;

 // Sample points along the path
 while (length < totalLength) {
 const point = commander.getPointAtLength(length);

```

```

 // Only add point if it's different from the last one
 if (!lastPoint || point.x !== lastPoint.x || point.y !== lastPoint.y) {
 points.push({ x: point.x, y: point.y });
 lastPoint = point;
 }

 length += sampleLength;
}

// Ensure we get the last point
const finalPoint = commander.getPointAtLength(totalLength);
if (lastPoint && (finalPoint.x !== lastPoint.x || finalPoint.y !== lastPoint.y)) {
 points.push({ x: finalPoint.x, y: finalPoint.y });
}

return points;
}

function calculatePosition(
 value: number | string | undefined,
 containerSize: number,
 elementSize: number
) {
 if (typeof value === "string" && value.endsWith("%")) {
 const percentage = parseFloat(value) / 100;
 return containerSize * percentage;
 }
 return typeof value === "number"
 ? value
 : elementSize - containerSize + elementSize / 2;
}

type GravityProps = {
 children: ReactNode
 debug?: boolean
 gravity?: { x: number; y: number }
 resetOnResize?: boolean
 grabCursor?: boolean
 addTopWall?: boolean
 autoStart?: boolean
 className?: string
}

```

```
type PhysicsBody = {
 element: HTMLElement
 body: Matter.Body
 props: MatterBodyProps
}
```

```
type MatterBodyProps = {
 children: ReactNode
 matterBodyOptions?: Matter.IBodyDefinition
 isDraggable?: boolean
 bodyType?: "rectangle" | "circle" | "svg"
 sampleLength?: number
 x?: number | string
 y?: number | string
 angle?: number
 className?: string
}
```

```
export type GravityRef = {
 start: () => void
 stop: () => void
 reset: () => void
}
```

```
const GravityContext = createContext<{
 registerElement: (
 id: string,
 element: HTMLElement,
 props: MatterBodyProps
) => void
 unregisterElement: (id: string) => void
} | null>(null)
```

```
const MatterBody = ({
 children,
 className,
 matterBodyOptions = {
 friction: 0.1,
 restitution: 0.1,
 density: 0.001,
 isStatic: false,
 },
 bodyType = "rectangle",
 isDraggable = true,
```

```

 sampleLength = 15,
 x = 0,
 y = 0,
 angle = 0,
 ...props
 }: MatterBodyProps) => {
 const elementRef = useRef<HTMLDivElement>(null)
 const idRef = useRef(Math.random().toString(36).substring(7))
 const context = useContext(GravityContext)

 useEffect(() => {
 if (!elementRef.current || !context) return
 context.registerElement(idRef.current, elementRef.current, {
 children,
 matterBodyOptions,
 bodyType,
 sampleLength,
 isDraggable,
 x,
 y,
 angle,
 ...props,
 })

 return () => context.unregisterElement(idRef.current)
 }, [props, children, matterBodyOptions, isDraggable])

 return (
 <div
 ref={elementRef}
 className={cn(
 "absolute",
 className,
 isDraggable && "pointer-events-none"
)}
 >
 {children}
 </div>
)
 }
}

const Gravity = forwardRef<GravityRef, GravityProps>(
 (
 {

```

```

 children,
 debug = false,
 gravity = { x: 0, y: 1 },
 grabCursor = true,
 resetOnResize = true,
 addTopWall = true,
 autoStart = true,
 className,
 ...props
 },
 ref
) => {
 const canvas = useRef<HTMLDivElement>(null)
 const engine = useRef(Engine.create())
 const render = useRef<Render>()
 const runner = useRef<Runner>()
 const bodiesMap = useRef(new Map<string, PhysicsBody>())
 const frameId = useRef<number>()
 const mouseConstraint = useRef<Matter.MouseConstraint>()
 const mouseDown = useRef(false)
 const [canvasSize, setCanvasSize] = useState({ width: 0, height: 0 })

 const isRunning = useRef(false)

 // Register Matter.js body in the physics world
 const registerElement = useCallback(
 (id: string, element: HTMLElement, props: MatterBodyProps) => {
 if (!canvas.current) return
 const width = element.offsetWidth
 const height = element.offsetHeight
 const canvasRect = canvas.current!.getBoundingClientRect()

 const angle = (props.angle || 0) * (Math.PI / 180)

 const x = calculatePosition(props.x, canvasRect.width, width)
 const y = calculatePosition(props.y, canvasRect.height, height)

 let body
 if (props.bodyType === "circle") {
 const radius = Math.max(width, height) / 2
 body = Bodies.circle(x, y, radius, {
 ...props.matterBodyOptions,
 angle: angle,
 render: {

```

```

 fillStyle: debug ? "#888888" : "#00000000",
 strokeStyle: debug ? "#333333" : "#00000000",
 lineWidth: debug ? 3 : 0,
 },
 })
} else if (props.bodyType === "svg") {
 const paths = element.querySelectorAll("path")
 const vertexSets: Matter.Vector[][] = []

 paths.forEach((path) => {
 const d = path.getAttribute("d")
 const p = parsePathToVertices(d!, props.sampleLength)
 vertexSets.push(p)
 })

 body = Bodies.fromVertices(x, y, vertexSets, {
 ...props.matterBodyOptions,
 angle: angle,
 render: {
 fillStyle: debug ? "#888888" : "#00000000",
 strokeStyle: debug ? "#333333" : "#00000000",
 lineWidth: debug ? 3 : 0,
 },
 })
} else {
 body = Bodies.rectangle(x, y, width, height, {
 ...props.matterBodyOptions,
 angle: angle,
 render: {
 fillStyle: debug ? "#888888" : "#00000000",
 strokeStyle: debug ? "#333333" : "#00000000",
 lineWidth: debug ? 3 : 0,
 },
 })
}

if (body) {
 World.add(engine.current.world, [body])
 bodiesMap.current.set(id, { element, body, props })
}
},
[debug]
)

```

```

// Unregister Matter.js body from the physics world
const unregisterElement = useCallback((id: string) => {
 const body = bodiesMap.current.get(id)
 if (body) {
 World.remove(engine.current.world, body.body)
 bodiesMap.current.delete(id)
 }
}, [])

// Keep react elements in sync with the physics world
const updateElements = useCallback(() => {
 bodiesMap.current.forEach(({ element, body }) => {
 const { x, y } = body.position
 const rotation = body.angle * (180 / Math.PI)

 element.style.transform = `translate(${
 x - element.offsetWidth / 2
 }px, ${y - element.offsetHeight / 2}px) rotate(${rotation}deg)`
 })

 frameId.current = requestAnimationFrame(updateElements)
}, [])

const initializeRenderer = useCallback(() => {
 if (!canvas.current) return

 const height = canvas.current.offsetHeight
 const width = canvas.current.offsetWidth

 Common.setDecomp(require("poly-decomp"))

 engine.current.gravity.x = gravity.x
 engine.current.gravity.y = gravity.y

 render.current = Render.create({
 element: canvas.current,
 engine: engine.current,
 options: {
 width,
 height,
 wireframes: false,
 background: "#00000000",
 },
 })
}, [])

```

```

const mouse = Mouse.create(render.current.canvas)
mouseConstraint.current = MouseConstraint.create(engine.current, {
 mouse: mouse,
 constraint: {
 stiffness: 0.2,
 render: {
 visible: debug,
 },
 },
})

```

// Add walls

```

const walls = [
 // Floor
 Bodies.rectangle(width / 2, height + 10, width, 20, {
 isStatic: true,
 friction: 1,
 render: {
 visible: debug,
 },
 }),
]

```

// Right wall

```

Bodies.rectangle(width + 10, height / 2, 20, height, {
 isStatic: true,
 friction: 1,
 render: {
 visible: debug,
 },
}),

```

// Left wall

```

Bodies.rectangle(-10, height / 2, 20, height, {
 isStatic: true,
 friction: 1,
 render: {
 visible: debug,
 },
}),
]

```

const topWall = addTopWall

```

? Bodies.rectangle(width / 2, -10, width, 20, {

```



```
 isStatic: true,
 friction: 1,
 render: {
 visible: debug,
 },
 })
 : null
```

```
if (topWall) {
 walls.push(topWall)
}
```

```
const touchingMouse = () =>
 Query.point(
 engine.current.world.bodies,
 mouseConstraint.current?.mouse.position || { x: 0, y: 0 }
).length > 0
```

```
if (grabCursor) {
 Events.on(engine.current, "beforeUpdate", (event) => {
 if (canvas.current) {
 if (!mouseDown.current && !touchingMouse()) {
 canvas.current.style.cursor = "default"
 } else if (touchingMouse()) {
 canvas.current.style.cursor = mouseDown.current
 ? "grabbing"
 : "grab"
 }
 }
 })
}
```

```
canvas.current.addEventListener("mousedown", (event) => {
 mouseDown.current = true
```

```
 if (canvas.current) {
 if (touchingMouse()) {
 canvas.current.style.cursor = "grabbing"
 } else {
 canvas.current.style.cursor = "default"
 }
 }
})
```

```
canvas.current.addEventListener("mouseup", (event) => {
 mouseDown.current = false
```

```

 if (canvas.current) {
 if (touchingMouse()) {
 canvas.current.style.cursor = "grab"
 } else {
 canvas.current.style.cursor = "default"
 }
 }
 })
}

```

```

World.add(engine.current.world, [mouseConstraint.current, ...walls])

```

```

render.current.mouse = mouse

```

```

runner.current = Runner.create()
Render.run(render.current)
updateElements()
runner.current.enabled = false

```

```

if (autoStart) {
 runner.current.enabled = true
 startEngine()
}
}, [updateElements, debug, autoStart])

```

```

// Clear the Matter.js world
const clearRenderer = useCallback(() => {
 if (frameId.current) {
 cancelAnimationFrame(frameId.current)
 }
}

```

```

if (mouseConstraint.current) {
 World.remove(engine.current.world, mouseConstraint.current)
}

```

```

if (render.current) {
 Mouse.clearSourceEvents(render.current.mouse)
 Render.stop(render.current)
 render.current.canvas.remove()
}

```

```

if (runner.current) {
 Runner.stop(runner.current)
}

```

```

 }

 if (engine.current) {
 World.clear(engine.current.world, false)
 Engine.clear(engine.current)
 }

 bodiesMap.current.clear()
 }, [])

const handleResize = useCallback(() => {
 if (!canvas.current || !resetOnResize) return

 const newWidth = canvas.current.offsetWidth
 const newHeight = canvas.current.offsetHeight

 setCanvasSize({ width: newWidth, height: newHeight })

 // Clear and reinitialize
 clearRenderer()
 initializeRenderer()
}, [clearRenderer, initializeRenderer, resetOnResize])

const startEngine = useCallback(() => {
 if (runner.current) {
 runner.current.enabled = true

 Runner.run(runner.current, engine.current)
 }
 if (render.current) {
 Render.run(render.current)
 }
 frameld.current = requestAnimationFrame(updateElements)
 isRunning.current = true
}, [updateElements, canvasSize])

const stopEngine = useCallback(() => {
 if (!isRunning.current) return

 if (runner.current) {
 Runner.stop(runner.current)
 }
 if (render.current) {
 Render.stop(render.current)
 }

```

```

 }
 if (frameId.current) {
 cancelAnimationFrame(frameId.current)
 }
 isRunning.current = false
 }, [])

```

```

const reset = useCallback(() => {
 stopEngine()
 bodiesMap.current.forEach(({ element, body, props }) => {
 body.angle = props.angle || 0

 const x = calculatePosition(
 props.x,
 canvasSize.width,
 element.offsetWidth
)
 const y = calculatePosition(
 props.y,
 canvasSize.height,
 element.offsetHeight
)
 body.position.x = x
 body.position.y = y
 })
 updateElements()
 handleResize()
}, [])

```

```

useImperativeHandle(
 ref,
 () => ({
 start: startEngine,
 stop: stopEngine,
 reset,
 }),
 [startEngine, stopEngine]
)

```

```

useEffect(() => {
 if (!resetOnResize) return

```

```

 const debouncedResize = debounce(handleResize, 500)
 window.addEventListener("resize", debouncedResize)

```

```

 return () => {
 window.removeEventListener("resize", debouncedResize)
 debouncedResize.cancel()
 }
 }, [handleResize, resetOnResize])

 useEffect(() => {
 initializeRenderer()
 return clearRenderer
 }, [initializeRenderer, clearRenderer])

 return (
 <GravityContext.Provider value={{ registerElement, unregisterElement }}>
 <div
 ref={canvas}
 className={cn(className, "absolute top-0 left-0 w-full h-full")}
 {...props}
 >
 {children}
 </div>
 </GravityContext.Provider>
)
}
)

```

```

Gravity.displayName = "Gravity"
export { Gravity, MatterBody }

```

demo.tsx

```

import { Gravity, MatterBody } from "@components/ui/gravity";

```

```

function Preview() {
 return (
 <div className="w-full h-full min-h-[500px] flex flex-col relative font-azeretMono">
 <div className="pt-20 text-6xl sm:text-7xl md:text-8xl text-black w-full text-center
font-calendas italic">
 fancy
 </div>
 <p className="pt-4 text-base sm:text-xl md:text-2xl text-black w-full text-center">
 components made with:
 </p>
 <Gravity gravity={{ x: 0, y: 1 }} className="w-full h-full">

```

```

<MatterBody
 matterBodyOptions={{ friction: 0.5, restitution: 0.2 }}
 x="30%"
 y="10%"
>
 <div className="text-xl sm:text-2xl md:text-3xl bg-[#0015ff] text-white rounded-full
hover:cursor-pointer px-8 py-4">
 react
 </div>
</MatterBody>
<MatterBody
 matterBodyOptions={{ friction: 0.5, restitution: 0.2 }}
 x="30%"
 y="30%"
>
 <div className="text-xl sm:text-2xl md:text-3xl bg-[#E794DA] text-white rounded-full
hover:cursor-grab px-8 py-4 ">
 typescript
 </div>
</MatterBody>
<MatterBody
 matterBodyOptions={{ friction: 0.5, restitution: 0.2 }}
 x="40%"
 y="20%"
 angle={10}
>
 <div className="text-xl sm:text-2xl md:text-3xl bg-[#1f464d] text-white rounded-full
hover:cursor-grab px-8 py-4 ">
 motion
 </div>
</MatterBody>
<MatterBody
 matterBodyOptions={{ friction: 0.5, restitution: 0.2 }}
 x="75%"
 y="10%"
>
 <div className="text-xl sm:text-2xl md:text-3xl bg-[#ff5941] text-white [#E794DA]
rounded-full hover:cursor-grab px-8 py-4 ">
 tailwind
 </div>
</MatterBody>
<MatterBody
 matterBodyOptions={{ friction: 0.5, restitution: 0.2 }}
 x="80%"

```

```

 y="20%"
 >
 <div className="text-xl sm:text-2xl md:text-3xl bg-orange-500 text-white [#E794DA]
rounded-full hover:cursor-grab px-8 py-4 ">
 drei
 </div>
 </MatterBody>
 <MatterBody
 matterBodyOptions={{ friction: 0.5, restitution: 0.2 }}
 x="50%"
 y="10%"
 >
 <div className="text-xl sm:text-2xl md:text-3xl bg-[#ffd726] text-white [#E794DA]
rounded-full hover:cursor-grab px-8 py-4 ">
 matter-js
 </div>
 </MatterBody>
 </Gravity>
 </div>
);
}

export { Preview };

...

```

Install NPM dependencies:

```

```bash
lodash, matter-js, poly-decomp, svg-path-commander
```

```

### Implementation Guidelines

1. Analyze the component structure and identify all required dependencies
2. Review the component's arguments and state
3. Identify any required context providers or hooks and install them
4. Questions to Ask
  - What data/props will be passed to this component?
  - Are there any specific state management requirements?
  - Are there any required assets (images, icons, etc.)?
  - What is the expected responsive behavior?
  - What is the best place to use this component in the app?

### Steps to integrate

0. Copy paste all the code above in the correct directories

1. Install external dependencies
2. Fill image assets with Unsplash stock images you know exist
3. Use lucide-react icons for svgs or logos if component requires them

## Profile cards

You are given a task to integrate an existing React component in the codebase

The codebase should support:

- shadcn project structure
- Tailwind CSS
- Typescript

If it doesn't, provide instructions on how to setup project via shadcn CLI, install Tailwind or Typescript.

Determine the default path for components and styles.

If default path for components is not /components/ui, provide instructions on why it's important to create this folder

Copy-paste this component to /components/ui folder:

```
``tsx
```

```
portfolio-hero.tsx
```

```
import React, { useState, useEffect, useRef, useMemo } from "react";
import { Menu, X, ChevronDown } from "lucide-react";
```

```
// Inline Button component
```

```
const Button = React.forwardRef<HTMLButtonElement,
React.ButtonHTMLAttributes<HTMLButtonElement>>(
 ({ className = "", children, ...props }, ref) => {
 return (
 <button
 ref={ref}
 className={`inline-flex items-center justify-center rounded-md px-4 py-2 text-sm
font-medium transition-colors ${className}`}
 {...props}
 >
 {children}
 </button>
);
 }
);
```

```
}
```



```

);
Button.displayName = "Button";

// BlurText animation component
interface BlurTextProps {
 text: string;
 delay?: number;
 animateBy?: "words" | "letters";
 direction?: "top" | "bottom";
 className?: string;
 style?: React.CSSProperties;
}

const BlurText: React.FC<BlurTextProps> = ({
 text,
 delay = 50,
 animateBy = "words",
 direction = "top",
 className = "",
 style,
}) => {
 const [inView, setInView] = useState(false);
 const ref = useRef<HTMLParagraphElement>(null);

 useEffect(() => {
 const observer = new IntersectionObserver(
 ([entry]) => {
 if (entry.isIntersecting) {
 setInView(true);
 }
 },
 { threshold: 0.1 }
);

 if (ref.current) {
 observer.observe(ref.current);
 }

 return () => {
 if (ref.current) {
 observer.unobserve(ref.current);
 }
 };
 }, []);

```

```

const segments = useMemo(() => {
 return animateBy === "words" ? text.split(" ") : text.split("");
}, [text, animateBy]);

return (
 <p ref={ref} className={`inline-flex flex-wrap ${className}`} style={style}>
 {segments.map((segment, i) => (
 <span
 key={i}
 style={{
 display: "inline-block",
 filter: inView ? "blur(0px)" : "blur(10px)",
 opacity: inView ? 1 : 0,
 transform: inView ? "translateY(0)" : `translateY(${direction === "top" ? "-20px" :
"20px"})`,
 transition: `all 0.5s ease-out ${i * delay}ms`,
 }}
 >
 {segment}
 {animateBy === "words" && i < segments.length - 1 ? "\u00A0" : ""}

))}
 </p>
);
};

```

```

export default function Component() {
 const [isDark, setIsDark] = useState(true);
 const [isMenuOpen, setIsMenuOpen] = useState(false);
 const menuRef = useRef<HTMLDivElement>(null);
 const buttonRef = useRef<HTMLButtonElement>(null);

```

```

 useEffect(() => {
 document.documentElement.classList.add("dark");
 }, []);

```

```

 useEffect(() => {
 const handleClickOutside = (event: MouseEvent) => {
 if (
 isMenuOpen &&
 menuRef.current &&
 buttonRef.current &&
 !menuRef.current.contains(event.target as Node) &&

```

```

 !buttonRef.current.contains(event.target as Node)
) {
 setIsMenuOpen(false);
 }
 };

 document.addEventListener("mousedown", handleClickOutside);
 return () => document.removeEventListener("mousedown", handleClickOutside);
 }, [isMenuOpen]);

const toggleTheme = () => {
 const newTheme = !isDark;
 setIsDark(newTheme);
 if (newTheme) {
 document.documentElement.classList.add("dark");
 } else {
 document.documentElement.classList.remove("dark");
 }
};

const menuItems = [
 { label: "HOME", href: "#", highlight: true },
 { label: "ABOUT", href: "#" },
 { label: "PROJECTS", href: "#" },
 { label: "EXPERIENCE", href: "#" },
 { label: "EDUCATION", href: "#" },
 { label: "WRITING", href: "#" },
 { label: "CONTACT", href: "#" },
];

return (
 <div
 className="min-h-screen text-foreground transition-colors"
 style={{
 backgroundColor: isDark ? "hsl(0 0% 0%)" : "hsl(0 0% 98%)",
 color: isDark ? "hsl(0 0% 100%)" : "hsl(0 0% 10%)",
 }}
 >
 { /* Header */ }
 <header className="fixed top-0 left-0 right-0 z-50 px-6 py-6">
 <nav className="flex items-center justify-between max-w-screen-2xl mx-auto">
 { /* Menu Button */ }
 <div className="relative">
 <button

```

```

 ref={buttonRef}
 type="button"
 className="p-2 transition-colors duration-300 z-50 text-neutral-500 hover:text-black
dark:hover:text-white"
 aria-label={isMenuOpen ? "Close menu" : "Open menu"}
 onClick={() => setIsMenuOpen(!isMenuOpen)}
 >
 {isMenuOpen ? (
 <X className="w-8 h-8 transition-colors duration-300" strokeWidth={2} />
) : (
 <Menu className="w-8 h-8 transition-colors duration-300" strokeWidth={2} />
)}
 </button>

 {isMenuOpen && (
 <div
 ref={menuRef}
 className="absolute top-full left-0 w-[200px] md:w-[240px] border-none shadow-2xl
mt-2 ml-4 p-4 rounded-lg z-[100]"
 style={{
 backgroundColor: isDark ? "hsl(0 0% 0%)" : "hsl(0 0% 98%)",
 }}
 >
 {menuItems.map((item) => (
 <a
 key={item.label}
 href={item.href}
 className="block text-lg md:text-xl font-bold tracking-tight py-1.5 px-2
cursor-pointer transition-colors duration-300"
 style={{
 color: item.highlight ? "#C3E41D" : isDark ? "hsl(0 0% 100%)" : "hsl(0 0% 10%)",
 }}
 onMouseEnter={(e) => {
 e.currentTarget.style.color = "#C3E41D";
 }}
 onMouseLeave={(e) => {
 e.currentTarget.style.color = item.highlight ? "#C3E41D" : (isDark ? "hsl(0 0%
100%)" : "hsl(0 0% 10%)");
 }}
 onClick={() => setIsMenuOpen(false)}
 >
 {item.label}

))}
)}

```

```

 </div>
 })
</div>

 {/ * Signature */}
 <div className="text-4xl" style={{ color: isDark ? "hsl(0 0% 100%)" : "hsl(0 0% 10%)",
fontFamily: "'Brush Script MT', 'Lucida Handwriting', cursive" }}>
 A
 </div>

 {/ * Theme Toggle */}
 <button
 type="button"
 onClick={toggleTheme}
 className="relative w-16 h-8 rounded-full hover:opacity-80 transition-opacity"
 style={{ backgroundColor: isDark ? "hsl(0 0% 15%)" : "hsl(0 0% 90%)" }}
 aria-label="Toggle theme"
 >
 <div
 className="absolute top-1 left-1 w-6 h-6 rounded-full transition-transform
duration-300"
 style={{
 backgroundColor: isDark ? "hsl(0 0% 100%)" : "hsl(0 0% 10%)",
 transform: isDark ? "translateX(2rem)" : "translateX(0)",
 }}
 />
 </button>
</nav>
</header>

 {/ * Hero Section */}
 <main className="relative min-h-screen flex flex-col">
 {/ * Centered Main Name - Always Perfectly Centered */}
 <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-full px-4">
 <div className="relative text-center">
 <div>
 <BlurText
 text="ALEX"
 delay={100}
 animateBy="letters"
 direction="top"
 className="font-bold text-[100px] sm:text-[140px] md:text-[180px] lg:text-[210px]
leading-[0.75] tracking-tighter uppercase justify-center whitespace-nowrap"
 style={{ color: "#C3E41D", fontFamily: "'Fira Code', monospace" }}

```

```

 />
 </div>
 <div>
 <BlurText
 text="KANE"
 delay={100}
 animateBy="letters"
 direction="top"
 className="font-bold text-[100px] sm:text-[140px] md:text-[180px] lg:text-[210px]
leading-[0.75] tracking-tighter uppercase justify-center whitespace-nowrap"
 style={{ color: "#C3E41D", fontFamily: "'Fira Code', monospace" }}
 />
 </div>

```

```

 { /* Profile Picture */ }
 <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 z-10">
 <div className="w-[65px] h-[110px] sm:w-[90px] sm:h-[152px] md:w-[110px]
md:h-[185px] lg:w-[129px] lg:h-[218px] rounded-full overflow-hidden shadow-2xl
transition-transform duration-300 hover:scale-110 cursor-pointer">

 </div>
 </div>
</div>
</div>

```

```

 { /* Tagline - Proper Distance Below Hero */ }
 <div className="absolute bottom-16 sm:bottom-20 md:bottom-24 lg:bottom-32
xl:bottom-36 left-1/2 -translate-x-1/2 w-full px-6">
 <div className="flex justify-center">
 <BlurText
 text="Designing human experiences in code."
 delay={150}
 animateBy="words"
 direction="top"
 className="text-[15px] sm:text-[18px] md:text-[20px] lg:text-[22px] text-center
transition-colors duration-300 text-neutral-500 hover:text-black dark:hover:text-white"
 style={{ fontFamily: "'Antic', sans-serif" }}
 />
 </div>
 </div>

```

```

 { /* Scroll Indicator */}
 <button
 type="button"
 className="absolute bottom-6 md:bottom-10 left-1/2 -translate-x-1/2 transition-colors
duration-300"
 aria-label="Scroll down"
 >
 <ChevronDown className="w-5 h-5 md:w-8 md:h-8 text-neutral-500 hover:text-black
dark:hover:text-white transition-colors duration-300" />
 </button>
 </main>
</div>
);
}

```

```

demo.tsx
import React from "react";
import Component from "@components/ui/portfolio-hero";

```

```

export default function Demo() {
 return (
 <>
 <link
 rel="stylesheet"

```

```

href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@700&family=Antic&display=s
wap"
 />
 <div className="w-full">
 <Component />
 </div>
 </>
);
}
...

```

Install NPM dependencies:

```

```bash
lucide-react
```

```

Implementation Guidelines

1. Analyze the component structure and identify all required dependencies

2. Review the component's arguments and state
3. Identify any required context providers or hooks and install them
4. Questions to Ask
  - What data/props will be passed to this component?
  - Are there any specific state management requirements?
  - Are there any required assets (images, icons, etc.)?
  - What is the expected responsive behavior?
  - What is the best place to use this component in the app?

#### Steps to integrate

0. Copy paste all the code above in the correct directories
1. Install external dependencies
2. Fill image assets with Unsplash stock images you know exist
3. Use lucide-react icons for svgs or logos if component requires them